

Checklist de Validação do Repositório antes da Entrega

Este checklist deve ser usado pela equipe antes de entregar o repositório para avaliação. A proposta é garantir que o projeto esteja organizado, documentado, testável e com evidências reais de desenvolvimento colaborativo.

1. README do projeto

Antes da entrega, verifique se o `README.md` permite que outra pessoa consiga entender, instalar, configurar, executar e testar o projeto sem depender de explicações externas.

- 1.1. O README apresenta o nome do projeto e uma descrição clara do problema resolvido.
- 1.2. O README informa o objetivo geral do sistema.
- 1.3. O README descreve as principais funcionalidades implementadas.
- 1.4. O README apresenta as tecnologias utilizadas, como Node.js, NestJS, TypeScript, PostgreSQL, Jest, Docker ou outras usadas no projeto.
- 1.5. O README informa os pré-requisitos para executar o projeto, incluindo versões recomendadas quando necessário.
- 1.6. O README mostra como clonar o repositório.
- 1.7. O README mostra como instalar as dependências do projeto.
- 1.8. O README explica como configurar o arquivo `.env`.
- 1.9. O README lista as variáveis de ambiente necessárias.
- 1.10. O README apresenta um exemplo de `.env`, sem expor dados sensíveis reais.
- 1.11. O README mostra como executar o projeto localmente.
- 1.12. O README mostra como rodar os testes.
- 1.13. O README mostra como gerar relatório de cobertura de testes, quando aplicável.
- 1.14. O README apresenta os principais endpoints implementados.
- 1.15. O README informa exemplos de requisição e resposta em JSON para os endpoints principais.
- 1.16. O README informa os principais códigos de status HTTP esperados.

- 1.17. O README contém seção de licença ou informa claramente a situação da licença do projeto.
 - 1.18. O README contém badges/shields úteis, quando aplicável, como status de build, versão do Node.js, licença ou testes.
 - 1.19. O README contém instruções de contribuição ou fluxo básico de colaboração, quando aplicável.
 - 1.20. O README está escrito de forma clara, organizada e sem erros graves de ortografia.
-

2. Pasta `docs/` e documentação técnica

A pasta `docs/` deve reunir a documentação construída ao longo do desenvolvimento. Ela não deve existir apenas como formalidade.

- 2.1. Existe uma pasta `docs/` no repositório.
 - 2.2. A pasta `docs/` contém a análise inicial do sistema.
 - 2.3. A documentação apresenta o problema que o sistema pretende resolver.
 - 2.4. A documentação apresenta o contexto do domínio do projeto.
 - 2.5. A documentação contém os casos de uso trabalhados na disciplina.
 - 2.6. A documentação contém as regras de negócio do sistema.
 - 2.7. As regras de negócio estão descritas de forma objetiva e verificável.
 - 2.8. A documentação contém o contrato da API ou referência clara aos endpoints.
 - 2.9. A documentação apresenta a arquitetura do projeto.
 - 2.10. A documentação explica a organização das camadas, módulos, controllers, services, repositories, DTOs e entities, quando aplicável.
 - 2.11. A documentação contém evidências de testes, execução ou validação, quando solicitado.
 - 2.12. A documentação está coerente com o código realmente implementado.
 - 2.13. Arquivos como `overview.md`, `architecture.md`, `api-contract.md`, `technical-documentation.md` ou equivalentes possuem conteúdo útil e atualizado.
 - 2.14. A documentação não está vazia, incompleta ou composta apenas por títulos sem desenvolvimento.
-

3. Casos de uso e regras de negócio

O projeto deve evidenciar que a implementação foi orientada pelos casos de uso e pelas regras de negócio definidas.

- 3.1. Os casos de uso estão identificados na documentação.
 - 3.2. Cada caso de uso possui objetivo claro.
 - 3.3. Cada caso de uso possui fluxo principal descrito.
 - 3.4. Cada caso de uso possui fluxos alternativos ou cenários de erro, quando aplicável.
 - 3.5. As regras de negócio associadas aos casos de uso estão documentadas.
 - 3.6. O código implementado respeita as regras de negócio documentadas.
 - 3.7. Os testes cobrem as regras de negócio mais importantes.
 - 3.8. Os endpoints implementados correspondem aos casos de uso planejados.
 - 3.9. Não há funcionalidades implementadas sem relação clara com a documentação do projeto.
-

4. Estrutura e organização do código

A estrutura do projeto deve facilitar leitura, manutenção e evolução.

- 4.1. O projeto possui estrutura de pastas organizada.
- 4.2. O código está separado por responsabilidade.
- 4.3. Controllers não concentram regras de negócio indevidamente.
- 4.4. Services concentram a lógica de aplicação ou domínio esperada.
- 4.5. Repositories ou camadas de persistência estão separados da lógica de negócio, quando aplicável.
- 4.6. DTOs estão organizados e utilizados corretamente.
- 4.7. Entities ou models estão organizados e coerentes com o domínio.
- 4.8. O projeto evita arquivos excessivamente grandes e difíceis de manter.
- 4.9. Os nomes de arquivos, classes, métodos e variáveis são claros.
- 4.10. O projeto não contém arquivos desnecessários versionados, como `node_modules`, `dist`, `.env` real, arquivos temporários ou lixo de IDE.

4.11. O `.gitignore` está configurado corretamente.

5. Testes automatizados

Não basta citar TDD, Jest ou testes no README. O repositório precisa conter testes reais, executáveis e coerentes com o sistema.

- 5.1. Existem arquivos de teste no projeto.
 - 5.2. Os testes estão versionados no repositório.
 - 5.3. Há testes unitários para services ou regras de negócio principais.
 - 5.4. Há testes para cenários de sucesso.
 - 5.5. Há testes para cenários de erro.
 - 5.6. Há testes para validações importantes.
 - 5.7. Há testes relacionados aos casos de uso implementados.
 - 5.8. Os testes conseguem ser executados com o comando informado no README.
 - 5.9. O comando de testes funciona em ambiente limpo após instalação das dependências.
 - 5.10. O projeto não possui testes quebrados na entrega.
 - 5.11. O README informa como executar os testes.
 - 5.12. Quando houver cobertura, o README informa como gerar o relatório de cobertura.
 - 5.13. Os testes têm nomes claros, indicando o comportamento esperado.
 - 5.14. Os testes não existem apenas como arquivos vazios ou exemplos sem relação com o projeto.
-

6. Testes manuais com REST Client ou equivalente

Quando o projeto usa arquivos `.http`, Postman, Insomnia ou ferramenta semelhante, esses artefatos devem permitir validação prática da API.

- 6.1. Existe arquivo de testes manuais, como `.http`, coleção Postman ou equivalente.
- 6.2. O arquivo contém cenários organizados por caso de uso ou recurso.
- 6.3. Os endpoints usam variáveis como `baseUrl`, `contentType` e identificadores reaproveitáveis, quando aplicável.

6.4. Há cenários de sucesso.

6.5. Há cenários de erro, como `400`, `404`, `403` ou outros relevantes.

6.6. As requisições possuem corpos JSON válidos.

6.7. O arquivo está atualizado com os endpoints realmente implementados.

6.8. O README explica como usar o arquivo de testes manuais.

7. Endpoints e contrato da API

A API deve estar clara para quem for consumir ou avaliar o projeto.

7.1. Os endpoints implementados estão documentados.

7.2. Cada endpoint possui método HTTP correto.

7.3. Cada endpoint possui rota clara e padronizada.

7.4. Os parâmetros de rota estão documentados, quando existirem.

7.5. Os parâmetros de query estão documentados, quando existirem.

7.6. O corpo da requisição está documentado, quando existir.

7.7. A resposta esperada está documentada.

7.8. Os códigos HTTP de sucesso e erro estão documentados.

7.9. Os endpoints retornam respostas coerentes com o contrato documentado.

7.10. Há consistência entre nomes de rotas, controllers, services e documentação.

7.11. Quando existir Swagger ou documentação automática, o README informa o caminho de acesso, como `/api/docs`.

8. Banco de dados, persistência e configuração

A configuração de persistência precisa estar clara e reproduzível.

8.1. O projeto informa qual banco de dados utiliza.

8.2. O README explica como configurar o banco localmente, quando aplicável.

8.3. O `.env.example` contém as variáveis relacionadas ao banco.

8.4. O projeto não versiona credenciais reais.

8.5. Há instruções para executar migrations, seed ou scripts de inicialização, quando existirem.

8.6. As entidades ou tabelas estão coerentes com o domínio documentado.

8.7. O projeto evita dependência de dados locais não documentados.

8.8. Se houver Docker ou Docker Compose, o README explica como utilizá-los.

9. Branches

As branches devem demonstrar organização do trabalho e rastreabilidade com as tarefas desenvolvidas.

9.1. As branches seguem um padrão de nomenclatura.

9.2. As branches usam prefixos adequados, como `feature/`, `fix/`, `docs/`, `test/` ou `refactor/`.

9.3. As branches fazem referência ao caso de uso, funcionalidade ou número da Issue, quando aplicável.

9.4. Não há branches com nomes genéricos, como `ultimos-ajustes`, `teste`, `final`, `ajustes` ou `nova-branch`.

9.5. As branches foram criadas a partir da `main` atualizada.

9.6. Branches antigas ou já integradas foram removidas, quando apropriado.

9.7. A branch principal está estável no momento da entrega.

Exemplos de bons nomes de branch:

```
feature/uc03-return-loan
fix/uc03-return-validation
test/uc03-rest-client
docs/update-readme-setup
refactor/uc03-service-structure
```

10. Commits

As mensagens de commit devem permitir entender o histórico do desenvolvimento.

10.1. Os commits possuem mensagens claras e específicas.

10.2. Os commits indicam o tipo de alteração realizada.

10.3. Os commits evitam mensagens genéricas, como `ajustes`, `alterações`, `final`, `pequena alteração` ou `últimos ajustes`.

10.4. As mensagens estão escritas com cuidado e sem erros graves.

10.5. Os commits estão relacionados às alterações realmente feitas nos arquivos.

10.6. Os commits são pequenos e focados em uma alteração lógica.

10.7. Quando aplicável, os commits fazem referência à Issue ou ao caso de uso.

10.8. O histórico não concentra muitas alterações diferentes em um único commit genérico.

Exemplos de boas mensagens de commit:

```
feat(uc03): implement return loan endpoint
test(uc03): add REST Client scenarios for loan return
docs(readme): add setup and test instructions
refactor(uc03): reorganize return loan service
fix(uc03): validate loan status before return
```

11. Issues no GitHub

As Issues devem demonstrar planejamento, acompanhamento e rastreabilidade do trabalho.

11.1. Existem Issues para os casos de uso ou tarefas principais.

11.2. As Issues possuem títulos claros.

11.3. As Issues descrevem o que deve ser feito.

11.4. As Issues possuem critérios de aceite ou checklist interno, quando aplicável.

11.5. As Issues usam labels adequadas, como `backend`, `api`, `service`, `controller`, `database`, `testing`, `tdd:red`, `tdd:green` ou `tdd:refactor`.

11.6. As Issues estão relacionadas a branches, commits ou Pull Requests.

11.7. As Issues fechadas correspondem a entregas realmente concluídas.

11.8. As Issues possuem comentários ou evidências do que foi feito, quando necessário.

11.9. As tarefas foram divididas em partes menores e compreensíveis.

11.10. Não há Issues fechadas sem implementação correspondente no código.

12. Pull Requests e revisão

O Pull Request deve evidenciar integração organizada do trabalho e revisão mínima da equipe.

12.1. As alterações principais foram integradas por Pull Request, quando a equipe usou esse fluxo.

12.2. O título do Pull Request é claro.

12.3. A descrição do Pull Request resume o que foi implementado.

12.4. O Pull Request referencia a Issue relacionada.

12.5. O Pull Request apresenta evidências de teste ou validação.

12.6. O Pull Request foi revisado por outro integrante, quando aplicável.

12.7. Comentários de revisão foram respondidos ou resolvidos.

12.8. O Pull Request não mistura muitas funcionalidades sem relação.

12.9. O merge foi realizado de forma organizada.

13. Participação da equipe

O repositório deve evidenciar participação real dos integrantes, especialmente em projetos em equipe.

13.1. Os integrantes da equipe estão identificados no README ou na documentação.

13.2. Há commits de mais de um integrante, quando o trabalho foi em equipe.

13.3. Há participação em Issues por diferentes integrantes.

13.4. Há participação em Pull Requests por diferentes integrantes.

13.5. Há evidências de revisão de código.

13.6. A divisão de papéis está clara, como Owner/Admin, implementador e revisor.

13.7. A participação não está concentrada em apenas uma pessoa, salvo justificativa documentada.

13.8. A equipe consegue explicar a contribuição individual de cada membro.

13.9. O histórico do GitHub é compatível com a equipe informada.

14. Qualidade geral da entrega

Antes de entregar, faça uma última verificação geral.

14.1. O projeto instala corretamente em ambiente limpo.

14.2. O projeto executa sem erros imediatos.

14.3. Os principais endpoints funcionam.

14.4. Os testes automatizados passam.

14.5. Os testes manuais foram conferidos.

14.6. A documentação está atualizada.

14.7. O README está completo.

14.8. A pasta `docs/` contém conteúdo relevante.

14.9. As Issues, branches, commits e PRs contam uma história coerente do desenvolvimento.

14.10. Não há arquivos sensíveis versionados.

14.11. Não há arquivos desnecessários versionados.

14.12. A entrega representa o que foi solicitado na disciplina.

15. Checklist final rápido antes de enviar o link

Use esta seção como conferência final no dia da entrega.

15.1. O link do repositório está correto e acessível ao professor.

15.2. O repositório não está privado sem permissão de acesso para avaliação.

15.3. A branch `main` contém a versão final da entrega.

15.4. O README explica como clonar, instalar, configurar `.env`, executar e testar.

15.5. Existe `.env.example` e não existe `.env` real versionado.

15.6. A pasta `docs/` contém análise, casos de uso e regras de negócio.

15.7. Existem testes automatizados reais.

15.8. Existem evidências de testes manuais da API, quando solicitado.

15.9. As Issues estão coerentes com o que foi implementado.

15.10. As branches têm nomes adequados.

15.11. Os commits têm mensagens claras.

15.12. A participação da equipe está visível no histórico do GitHub.

15.13. A entrega foi revisada por pelo menos outro integrante da equipe.

Observação final

Um bom repositório não é apenas aquele cujo código funciona. Ele precisa demonstrar organização, rastreabilidade, documentação, testes e colaboração. A avaliação deve considerar tanto o produto final quanto o caminho percorrido pela equipe durante o desenvolvimento.